

hodinám). Časy na tahy protihráčů jsme ve výpočtu neuváděli, protože ti mají podle pravidel tah provést do 55 sekund, což je doba, která se bez problémů „vejde“ do dostupného intervalu 120 sekund.

4.2.3 Programové konstrukce založené na slovech `async` a `await`

Pokusme se nyní naprogramovat úlohu, která bude reprezentována *korutinou*. Samotná úloha bude získána transformací funkce nazvané `task` do korutiny. Takto ztransformovanou funkci se následně pokusíme zavolat z funkce `main`, ovšem s tím, že explicitně pomocí klíčového slova `await` uvedeme, že budeme chtít počkat na dokončení korutiny (slova `async` a `await` se vyskytují právě v takových párech):

```
# Využití standardní knihovny asyncio:
# - vytvoření asynchronní úlohy
# - spuštění této úlohy
# - čekání na dokončení této úlohy
# - prozatím nefunkční řešení kvůli způsobu volání main!

import asyncio

# funkce, která dokáže dočasně předat řízení přes await
async def task():
    print("task started")
    # asynchronní volání - interpret může vykonávat jinou činnost
    await asyncio.sleep(5)
    print("task finished")

# běžná funkce
def main():
    # jedna z možností spuštění úlohy
    task1 = asyncio.create_task(task())
    print("task created")

    # (problém) - čekání na dokončení úlohy
    await task1

    print("done")

main()
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-await-1.py

Tento zápis však ještě není z pohledu sémantiky korektní, na což nás interpreter programovacího jazyka Python upozorní po spuštění skriptu. Problém spočívá v tom, že konstrukci `await` lze použít pouze ve funkci ztransformované na korutinu klíčovým slovem `async`:

```
File "async-await-1.py", line 18
    await task1
    ^
SyntaxError: 'await' outside async function
```

4.2.4 Vytvoření korutiny z funkce `main`

Sémanticky korektní zápis skriptu vyžaduje, aby se i z funkce `main` nějakým způsobem vytvořila *korutina*. Korutinu ovšem nelze volat přímo, tedy zápisem `main()`, ale jen nepřímo s využitím knihovny funkce nazvané `asyncio.run` (ovšem jak uvidíme v dalších kapitolách, máme ve skutečnosti k dispozici i některé další možnosti):

```
# Využití standardní knihovny asyncio:
# - vytvoření asynchronní úlohy
# - spuštění této úlohy
# - čekání na dokončení této úlohy
# - korektní volání funkce main

import asyncio

# funkce, která dokáže dočasně předat řízení přes await
async def task():
    print("task started")
    # asynchronní volání - interpret může vykonávat jinou činnost
    await asyncio.sleep(5)
    print("task finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    # jedna z možností spuštění úlohy
    task1 = asyncio.create_task(task())
    print("task created")
```

```
# čekání na dokončení úlohy
await task1

print("done")

# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-await-2.py

Nyní by již výsledek měl odpovídat očekávání:

```
task created
task started
<nyní se běh na přibližně pět sekund zastaví>
task finished
done
```

4.2.5 Vytvoření, spuštění a čekání na dokončení dvou korutin

V dalším demonstračním příkladu je ukázáno, jakým způsobem je možné vytvořit a ihned poté spustit dvě korutiny, které simulují nějakou smysluplnou činnost zavoláním `asyncio.sleep(čas)`, což umožní, aby interpret programovacího jazyka Python mezitím spustil jiný kód. Na dokončení obou korutin se čeká v konstrukci `await` (přesněji řečeno se čeká na dokončení první korutiny a teprve poté na dokončení korutiny druhé, což v praxi nemusí být zcela ideální řešení):

```
# Využití standardní knihovny asyncio:
# - vytvoření dvou asynchronních úloh
# - spuštění obou těchto úloh
# - čekání na dokončení obou těchto úloh

import asyncio

# funkce, která dokáže dočasně předat řízení přes await
async def task(name):
    print(f"{name} task started")
    # asynchronní volání - interpret může vykonávat jinou činnost
```

```
    await asyncio.sleep(5)
    print(f"{name} task finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    # spuštění první úlohy
    task1 = asyncio.create_task(task("first"))
    print("first task created")

    # spuštění druhé úlohy
    task2 = asyncio.create_task(task("second"))
    print("second task created")

    # čekání na dokončení úloh
    await task1
    await task2

    print("done")

# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/async-await/async-await-3.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/async-await/async-await-3.py)

Po spuštění takto upraveného skriptu uvidíme na terminálu následující zprávy:

```
first task created
second task created
first task started
second task started
<nyní se běh na přibližně pět sekund zastaví>
first task finished
second task finished
done
```

Za povšimnutí stojí především fakt, že se běh skriptu zastaví na pět sekund, protože obě úlohy jsou skutečně do značné míry souběžné (*concurrent*). V případě, že bychom nepoužili korutiny (tedy dvojice `async` + `await`), trvalo by dokončení celého skriptu více než deset sekund. I toto tvr-

zení si můžeme velmi snadno ověřit změřením celkového času trvání obou úloh. To je realizováno v dalším příkladu:

```
# Využití standardní knihovny asyncio:
# - vytvoření dvou asynchronních úloh
# - spuštění obou těchto úloh
# - čekání na dokončení obou úloh
# - doba měření doby trvání běhu obou úloh

import asyncio
from time import perf_counter

# funkce, která dokáže dočasně předat řízení přes await
async def task(name):
    print(f"{name} task started")
    # asynchronní volání - interpret může vykonávat jinou činnost
    await asyncio.sleep(5)
    print(f"{name} task finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    # spuštění první úlohy
    task1 = asyncio.create_task(task("first"))
    print("first task created")

    # spuštění druhé úlohy
    task2 = asyncio.create_task(task("second"))
    print("second task created")

    # čekání na dokončení úloh
    await task1
    await task2

    print("done")

# měření času
start = perf_counter()
```

```
# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

```
# měření času
finish = perf_counter()
```

```
# výpis dosaženého času
print(f"Time elapsed: {finish-start : 5.1f}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-await-5.py

Výsledkem by nyní měl být čas, který dosahuje přibližně pěti sekund:

```
first task created
second task created
first task started
second task started
first task finished
second task finished
done
Time elapsed: 5.0
```

4.2.6 Časy doby běhu synchronní a asynchronní varianty programu

Porovnejme nyní dosažený čas se „synchronní“ variantou, v níž vynecháme klíčová slova `async` i `await` a bude se tedy jednat o běžné postupně spouštěné úlohy, nikoli o úlohy běžící souběžně:

```
# Synchronní běh:
# - vytvoření dvou ynnchronních úloh
# - spuštění dvou úloh (jedna po druhé)
# - čekání na dokončení obou úloh
# - doba měření doby trvání běhu obou úloh
```

```
from time import perf_counter, sleep
```

```
# běžná synchronní funkce
def task(name):
    print(f"{name} task started")
    # ynnchronní volání
```

```
        sleep(5)
        print(f"{name} task finished")

# běžná synchronní funkce
def main():
    # spuštění první úlohy
    task("first")

    # spuštění druhé úlohy
    task("second")

    print("done")

# měření času
start = perf_counter()

# přímé spuštění synchronní funkce
main()

# měření času
finish = perf_counter()

# výpis dosaženého času
print(f"Time elapsed: {finish-start : 5.1f}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/no-async-await.py

Nyní by měl běh celého takto upraveného skriptu trvat nikoli přibližně pět sekund, ale celých deset sekund (popř. nepatrně delší dobu):

```
first task started
first task finished
second task started
second task finished
done
Time elapsed: 10.0
```

Poznámka: prozatím jsme spouštěli jen dvojici úloh, které mohly běžet souběžně. Pokud by těchto úloh bylo více, může být urychlení běhu celého skriptu ještě výraznější.

4.2.7 Získání výsledků z korutin

Konstrukci založenou na klíčovém slovu `await` je možné v případě potřeby použít i pro získání (přečtení) výsledků z korutin, což je hodnota/hodnoty vrácené přes standardní programovou konstrukci `return`. Opět si ukažme velmi jednoduchý způsob použití. Upravíme předchozí demonstrační příklad do takové podoby, aby se v něm spustily tři korutiny, které současně vrací nějaký výsledek – konkrétně svoje jméno, ovšem zapsané zprava doleva (tím simulujeme nějaký smysluplný výpočet). V korutině představované funkcí `main` počkáme na dokončení všech korutin a vypíšeme jejich výsledky (tedy ona otočená jména):

```
# Využití standardní knihovny asyncio:
# - vytvoření tří asynchronních úloh
# - spuštění všech tří úloh
# - čekání na dokončení všech tří úloh
# - tisk výsledků získaných z úloh

import asyncio

# funkce, která dokáže dočasně předat řízení přes await
async def task(name):
    print(f"{name} task started")
    # asynchronní volání - interpret může vykonávat jinou činnost
    await asyncio.sleep(5)
    print(f"{name} task finished")

    # vrátíme hodnotu volající (čekající) funkci
    return name[::-1]

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    # spuštění první úlohy
    task1 = asyncio.create_task(task("first"))
    print("first task created")

    # spuštění druhé úlohy
```

```
task2 = asyncio.create_task(task("second"))
print("second task created")

# spuštění třetí úlohy
task3 = asyncio.create_task(task("third"))
print("third task created")

# čekání na dokončení úloh s výpisem vypočtených výsledků
print("result of task #1:", await task1)
print("result of task #2:", await task2)
print("result of task #3:", await task3)

print("done")

# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-await-4.py

Jednotlivé korutiny po určité době vrátí své jméno, ovšem znaky jsou uvedeny v opačném pořadí:

```
first task created
second task created
third task created
first task started
second task started
third task started
<nyní se běh na přibližně pět sekund zastaví>
first task finished
second task finished
third task finished
result of task #1: tsrif
result of task #2: dnoces
result of task #3: driht
done
```

4.2.8 Využití fronty ve funkci komunikačního kanálu mezi korutinami

Pro komunikaci mezi korutinami, je možné využít několika typů komunikačních prostředků. Velmi často se můžeme setkat s využitím front (*queue*), a to z toho důvodu, že tento komunikační prostředek programátorům nabízí velmi snadno pochopitelnou a použitelnou „sémantiku“ pro jednostrannou či oboustrannou komunikaci. Konkrétně se přitom jedná o instanci třídy `asyncio.Queue` (to je důležité – tato třída je totiž připravena pro asynchronní úlohy). Všechny operace s touto frontou, což jsou především operace *put* (přidání prvku na konec fronty) a *get* (přečtení prvku ze začátku fronty), se přitom musí volat v programové konstrukci `await`. Ukažme si to na (prozatím velmi jednoduchém) demonstračním příkladu, ve kterém se pro komunikaci používá právě výše zmíněná fronta `asyncio.Queue`:

```
# Využití standardní knihovny asyncio:
# - příprava dat pro úlohy
# - komunikace mezi úlohami přes frontu
# - spuštění asynchronních úloh
# - bez čekání na dokončení (zpětné přečtení výsledků)

import asyncio

# vykonání úloh, jejichž argumenty jsou přečteny z fronty
async def task(name, queue):
    print(f"\tCoroutine #{name} started")
    while not queue.empty():
        param = await queue.get()
        print(f"\t\tTask with argument {param : 5.2f} started")
        await asyncio.sleep(5)
        print(f"\t\tTask with argument {param : 5.2f} finished")
    print(f"\tCoroutine #{name} finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    print("Main started")
    queue = asyncio.Queue()

    for i in range(10):
        await queue.put(1 / (i + 1))
```

```
for n in range(1, 11):
    asyncio.create_task(task(f"{n}", queue))

print("Main finished")

# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/async-await/async-queue-1.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/async-await/async-queue-1.py)

Po spuštění tohoto skriptu se nejdříve do fronty vloží několik argumentů, které určují, jaká úloha se má vykonat, a následně se tyto úlohy spustí. Přitom se nečeká na jejich dokončení, což je ostatně dobře patrné z výsledku:

```
Main started
Main finished
  Coroutine #1 started
    Task with argument 1.00 started
  Coroutine #2 started
    Task with argument 0.50 started
  Coroutine #3 started
    Task with argument 0.33 started
  Coroutine #4 started
    Task with argument 0.25 started
  Coroutine #5 started
    Task with argument 0.20 started
  Coroutine #6 started
    Task with argument 0.17 started
  Coroutine #7 started
    Task with argument 0.14 started
  Coroutine #8 started
    Task with argument 0.12 started
  Coroutine #9 started
    Task with argument 0.11 started
  Coroutine #10 started
    Task with argument 0.10 started
```

Ze zpráv vypsaných na terminál je patrné, že se sice skutečně spustilo deset úloh (viz druhou smyčku zapsanou ve zdrojovém kódu) a dokonce se těmto úlohám správně předal argument, ov-

šem ihned poté byl skript ukončen aniž by se čekalo na dokončení těchto úloh. A další zajímavost – zpráva „Main finished“ vypsaná v rámci korutiny `main` je (většinou) vypsaná jako první, tedy ještě předtím, než korutiny získaly argumenty z fronty a začaly je zpracovávat.

4.2.9 Postupné (synchronní) spouštění úloh

Při spouštění úloh má programátor plně pod kontrolou, jestli se bude jednat o spuštění (a následný běh) asynchronní či synchronní. V dalším demonstračním příkladu si sice připravíme argumenty pro dvacet úloh, ale všechny budou vykonány postupně ve smyčce. V praxi tedy nastane situace, v níž ihned první zavolaná korutina zpracuje všechny připravené argumenty úloh a prakticky žádná část skriptu nebude provedena souběžně:

```
# Využití standardní knihovny asyncio:
# - příprava dat pro úlohy
# - komunikace mezi úlohami přes frontu
# - postupné spuštění asynchronních úloh s čekáním na dokončení

import asyncio

# vykonání úloh, jejichž argumenty jsou přečteny z fronty
async def task(name, queue):
    print(f"\tCoroutine #{name} started")
    while not queue.empty():
        param = await queue.get()
        print(f"\t\tTask with argument {param : 5.2f} started")
        await asyncio.sleep(5)
        print(f"\t\tTask with argument {param : 5.2f} finished")
    print(f"\tCoroutine #{name} finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    print("Main started")
    queue = asyncio.Queue()

    for i in range(10):
        await queue.put(1 / (i + 1))

    for n in range(1, 11):
        print(f"Starting coroutine #{n}")
```



```
await asyncio.gather(asyncio.create_task(task(f"{n}", queue)))

print("Main finished")

# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-queue-2.py

Po spuštění takto upraveného skriptu by se měly vypsat informace o tom, že veškerou práci ve skutečnosti vykonala pouze první korutina. Druhá korutina i všechny korutiny následující již měly k dispozici pouze prázdnou frontu, takže jejich smyčka `while` byla ihned ukončena (a tím pádem byl ukončen i jejich běh). Toto chování jasně vidíme z vypsaných zpráv:

```
Main started
Starting coroutine #1
  Coroutine #1 started
    Task with argument 1.00 started
    Task with argument 1.00 finished
    Task with argument 0.50 started
    Task with argument 0.50 finished
    Task with argument 0.33 started
    Task with argument 0.33 finished
    Task with argument 0.25 started
    Task with argument 0.25 finished
    Task with argument 0.20 started
    Task with argument 0.20 finished
    Task with argument 0.17 started
    Task with argument 0.17 finished
    Task with argument 0.14 started
    Task with argument 0.14 finished
    Task with argument 0.12 started
    Task with argument 0.12 finished
    Task with argument 0.11 started
    Task with argument 0.11 finished
    Task with argument 0.10 started
    Task with argument 0.10 finished
  Coroutine #1 finished
Starting coroutine #2
```

```
Coroutine #2 started
Coroutine #2 finished
Starting coroutine #3
Coroutine #3 started
Coroutine #3 finished
Starting coroutine #4
Coroutine #4 started
Coroutine #4 finished
Starting coroutine #5
Coroutine #5 started
Coroutine #5 finished
Starting coroutine #6
Coroutine #6 started
Coroutine #6 finished
Starting coroutine #7
Coroutine #7 started
Coroutine #7 finished
Starting coroutine #8
Coroutine #8 started
Coroutine #8 finished
Starting coroutine #9
Coroutine #9 started
Coroutine #9 finished
Starting coroutine #10
Coroutine #10 started
Coroutine #10 finished
Main finished
```

Tento skript bude dokončen za přibližně 50 sekund, protože se desetkrát bude volat funkce `sleep` s pětisekundovým intervalem a všechny tyto operace budou provedeny sekvenčně za sebou. To evidentně není příliš efektivním řešením, především v okamžiku, kdy se namísto `sleep` bude provádět nějaká smysluplná operace...

4.2.10 Asynchronní spouštění úloh ve větším množství korutin

Pro spuštění většího množství korutin s čekáním na jejich dokončení je možné použít funkci nazvanou `asyncio.gather`, které se předají jednotlivé úlohy (tedy korutiny, které se mají vykonat). Následuje příklad použití pěti korutin zpracovávajících dvacet argumentů získávaných (souběžně!) z jediné fronty:

— 4 Postupné rozšiřování syntaxe a sémantiky Pythonu

```
# Využití standardní knihovny asyncio:
# - příprava dat pro úlohy
# - komunikace mezi úlohami přes frontu
# - souběžné spuštění asynchronních úloh s čekáním na dokončení

import asyncio

# vykonání úloh, jejichž argumenty jsou přečteny z fronty
async def task(name, queue):
    print(f"\t\tCoroutine #{name} started")
    while not queue.empty():
        param = await queue.get()
        print(f"\t\tTask with argument {param : 5.2f} started")
        await asyncio.sleep(5)
        print(f"\t\tTask with argument {param : 5.2f} finished")
    print(f"\t\tCoroutine #{name} finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    print("Main started")
    queue = asyncio.Queue()

    for i in range(10):
        await queue.put(1 / (i + 1))

    # spuštění a čekání na dokončení úloh
    await asyncio.gather(
        asyncio.create_task(task(1, queue)),
        asyncio.create_task(task(2, queue)),
        asyncio.create_task(task(3, queue)),
        asyncio.create_task(task(4, queue)),
        asyncio.create_task(task(5, queue)),
    )
    print("Main finished")

# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-queue-3.py

Nyní se jednotlivé úlohy provádí částečně asynchronně (a tedy vlastně i souběžně), což je patrné i z výpisu prováděných operací:

```
Main started
  Coroutine #1 started
    Task with argument 1.00 started
  Coroutine #2 started
    Task with argument 0.50 started
  Coroutine #3 started
    Task with argument 0.33 started
  Coroutine #4 started
    Task with argument 0.25 started
  Coroutine #5 started
    Task with argument 0.20 started
    Task with argument 1.00 finished
    Task with argument 0.17 started
    Task with argument 0.50 finished
    Task with argument 0.14 started
    Task with argument 0.33 finished
    Task with argument 0.12 started
    Task with argument 0.25 finished
    Task with argument 0.11 started
    Task with argument 0.20 finished
    Task with argument 0.10 started
    Task with argument 0.17 finished
  Coroutine #1 finished
    Task with argument 0.14 finished
  Coroutine #2 finished
    Task with argument 0.12 finished
  Coroutine #3 finished
    Task with argument 0.11 finished
  Coroutine #4 finished
    Task with argument 0.10 finished
  Coroutine #5 finished
Main finished
```

Doba trvání běhu tohoto skriptu nepatrně přesahuje 10 sekund, protože se sice bude desetkrát volat operace `sleep` s pětisekundovými intervaly, ale vždy přibližně pět takových volání bude provedeno souběžně. To je již lepší výsledek, než tomu bylo v případě předchozího skriptu.

4.2.11 Asynchronní spouštění všech úloh, které poběží souběžně

Vzhledem k tomu, že se má vykonat celkem deset úloh, můžeme pro tuto činnost využít deseti korutin běžících souběžně. Tím dosáhneme toho, že operace `sleep`, kterými simulujeme reálnou úlohu, poběží skutečně prakticky po celou dobu souběžně, což by mělo značně zkrátit celkovou dobu trvání běhu skriptu:

```
# Využití standardní knihovny asyncio:
# - příprava dat pro úlohy
# - komunikace mezi úlohami přes frontu
# - souběžné spuštění asynchronních úloh s čekáním na dokončení

import asyncio

# vykonání úloh, jejichž argumenty jsou přečteny z fronty
async def task(name, queue):
    print(f"\tCoroutine #{name} started")
    while not queue.empty():
        param = await queue.get()
        print(f"\t\tTask with argument {param : 5.2f} started")
        await asyncio.sleep(5)
        print(f"\t\tTask with argument {param : 5.2f} finished")
    print(f"\tCoroutine #{name} finished")

# funkce, která dokáže dočasně předat řízení přes await
async def main():
    print("Main started")
    queue = asyncio.Queue()

    for i in range(10):
        await queue.put(1 / (i + 1))

    # spuštění a čekání na dokončení úloh
    await asyncio.gather(
        asyncio.create_task(task(1, queue)),
```

```
    asyncio.create_task(task(2, queue)),
    asyncio.create_task(task(3, queue)),
    asyncio.create_task(task(4, queue)),
    asyncio.create_task(task(5, queue)),
    asyncio.create_task(task(6, queue)),
    asyncio.create_task(task(7, queue)),
    asyncio.create_task(task(8, queue)),
    asyncio.create_task(task(9, queue)),
    asyncio.create_task(task(10, queue)),
)
print("Main finished")
```

```
# spuštění funkce, jenž je definována s async
asyncio.run(main())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/async-await/async-queue-4.py

Z výpisu je patrné, že se skutečně všech deset úloh spustilo souběžně v deseti korutinách:

```
Main started
  Coroutine #1 started
    Task with argument 1.00 started
  Coroutine #2 started
    Task with argument 0.50 started
  Coroutine #3 started
    Task with argument 0.33 started
  Coroutine #4 started
    Task with argument 0.25 started
  Coroutine #5 started
    Task with argument 0.20 started
  Coroutine #6 started
    Task with argument 0.17 started
  Coroutine #7 started
    Task with argument 0.14 started
  Coroutine #8 started
    Task with argument 0.12 started
  Coroutine #9 started
    Task with argument 0.11 started
  Coroutine #10 started
```

```
Task with argument 0.10 started
Task with argument 1.00 finished
Coroutine #1 finished
Task with argument 0.50 finished
Coroutine #2 finished
Task with argument 0.33 finished
Coroutine #3 finished
Task with argument 0.25 finished
Coroutine #4 finished
Task with argument 0.20 finished
Coroutine #5 finished
Task with argument 0.17 finished
Coroutine #6 finished
Task with argument 0.14 finished
Coroutine #7 finished
Task with argument 0.12 finished
Coroutine #8 finished
Task with argument 0.11 finished
Coroutine #9 finished
Task with argument 0.10 finished
Coroutine #10 finished
Main finished
```

Tato úloha bude dokončena za přibližně 5 sekund, resp. čas jejího běhu nepatrně tuto dobu přesáhne.

Poznámka: některé další možnosti souběžného a paralelního spouštění úloh si ukážeme v osmé kapitole.

4.3 Specifikace pozičních parametrů funkcí

Do programovacího jazyka Python byla ve verzi 3.8 přidána dvojice nových vlastností. Ty se, podobně jako v případě dalších v této kapitole představovaných nových vlastností, dotýkají jak *syntaxe*, tak i *sémantiky* tohoto programovacího jazyka. Představme si nyní první z těchto vlastností. Jedná se o možnost specifikace takzvaných *čistě pozičních* parametrů funkcí, což znamená, že argumenty pro tyto parametry je nutné předat bez uvedení jejich jména – toto chování tedy bude interpretrem Pythonu striktně vyžadováno.

4.3.1 Parametry a argumenty funkcí

Nejprve si však ujasněme, co je vlastně myšleno pod označením *parametr* a *argument*. Parametr resp. poněkud přesněji řečeno formální parametr je uváděn přímo v definici funkce. Například v následující funkci jsou specifikovány dva formální parametry nazvané *x* a *y*:

```
def add(x, y):
    return x+y
```

Při volání funkce naopak do volané funkce předáváme *argumenty* (které jsou přiřazeny parametrům v průběhu volání funkce). Opět si to ukažme na výše uvedené funkci *add*:

```
add(2, 3)
```

či:

```
add(1+len("foo"), 2*int("1234"))
```

Poznámka: mnohdy se však tato označení různým způsobem zaměňují, což většinou nevadí. V této kapitole se však budeme snažit o větší striktnost v jejich použití.

4.3.2 Volání funkce s pozičními parametry

To však není vše, protože programovací jazyk Python umožňuje, aby se při specifikaci argumentů volané funkce explicitně použila jména parametrů. Příkladem může být funkce *foo* z následujícího příkladu, kterou lze volat několika způsoby s tím, že se v obou ukázaných případech vždy provede naprosto stejný výpočet (což znamená, že je funkce v čase běhu volána se stejnými argumenty):

```
# Poziční a čistě poziční parametry funkcí:
# - definice a volání funkce s pozičními parametry

# funkce s pozičními parametry
def foo(x, y, z):
    return x + y - z

# volání funkce s předáním argumentů
print(foo(1, 2, 10))
```



```
# explicitní specifikace parametrů jménem
# se zachováním jejich pořadí
print(foo(x=1, y=2, z=10))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/positional-params/positional-params-1.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/positional-params/positional-params-1.py)

Díky tomu, že jsou argumenty této funkce explicitně pojmenovány, můžeme jejich pořadí při volání funkce prohodit. Ve skutečnosti ovšem budou argumenty do funkce předány ve správném (resp. očekávaném) pořadí, což znamená, že se přiřadí do správných parametrů:

```
# Poziční a čistě poziční parametry funkcí:
# - definice a volání funkce s pozičními parametry
# - explicitní pojmenování parametrů při volání funkce
```

```
# funkce s pozičními parametry
def foo(x, y, z):
    return x + y - z
```

```
# explicitní specifikace parametrů jménem
# bez zachování jejich pořadí
print(foo(z=1, y=2, x=10))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/positional-params/positional-params-2.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/positional-params/positional-params-2.py)

4.3.3 Čistě poziční parametry funkcí

Jak již bylo zmíněno na začátku této kapitoly, byla do Pythonu 3.8 přidána podpora pro takzvané čistě poziční parametry funkcí. Jedná se o nový způsob zápisu definice funkce, který umožňuje rozlišit argumenty, které je nutné při volání funkce zapisovat jen pozičně (tedy na základě jejich pořadí), od argumentů zapisovaných podle rozhodnutí programátora buď pozičně nebo se specifikací jména parametru (což jsme si ukázali výše). Při definici funkce je možné čistě poziční parametry odlišit od běžných parametrů zápisem speciálního *pseudoparametru* pojmenovaného `/` (což pochopitelně není korektní jméno parametru, neboť se jedná o rezervovaný znak se speciálními významy).

Podívejme se nyní na funkci, jejíž všechny tři parametry jsou definovány jako čistě poziční:

```
# Poziční a čistě poziční parametry funkcí:
# - definice a volání funkce s čistě pozičními parametry
# - pokus o zavolání funkce s pojmenováním parametrů
```

```
print(foo)

# funkce s čistě pozičními parametry
def foo(x, y, z, /):
    return x + y - z
```

```
print(foo)

# volání funkce s předáním argumentů
print(foo(1, 2, 10))
```

```
# explicitní specifikace parametrů jménem
# není povolena (nezávisle na pořadí)
print(foo(x=1, y=2, z=10))
print(foo(z=1, y=2, x=10))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/positional-params/positional-params-3.py

Takto definovanou funkci je možné zavolat pouze stylem `foo(výraz, výraz, výraz)` a nikoli pojmenováním argumentů. Z tohoto důvodu první volání funkce bude korektní a funkce vrátí hodnotu `-7`, ovšem druhé a třetí volání již korektní není a skript při svém běhu vyhodí výjimku `TypeError`:

```
Traceback
-7

Traceback (most recent call last):
  File "positional-params-3.py", line 13, in <module>
    print(foo(x=1, y=2, z=10))
TypeError: foo() got some positional-only arguments passed as keyword arguments: 'x, y, z'
```

4.3.4 Kombinace čistě pozičních parametrů s běžnými pozičními parametry

Pseudoparametr `/`, který v programovacím jazyku Python funguje jako oddělovač čistě pozičních parametrů od běžných parametrů, samozřejmě nemusí být uveden na posledním místě. Můžeme například chtít, aby první dva parametry byly čistě poziční a další (třetí) argument může být při volání funkce předán libovolným způsobem. Toto chování zajistíme takto:

```
# Poziční a čistě poziční parametry funkcí:
# - definice a volání funkce se dvěma čistě pozičními parametry

# funkce se dvěma čistě pozičními parametry
def foo(x, y, /, z):
    return x + y - z

# volání funkce s předáním argumentů
print(foo(1, 2, 10))

# explicitní specifikace parametru z jménem
# je povolena
print(foo(1, 2, z=10))

# explicitní specifikace parametrů x a y jménem
# není povolena (nezávisle na pořadí)
print(foo(x=1, y=2, z=10))
print(foo(z=1, y=2, x=10))
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/positional-params/positional-params-4.py

Z výsledků získaných spuštěním tohoto skriptu je zřejmé, že první dva zvolené způsoby volání funkce jsou korektní a další dva způsoby nekorektní:

```
-7
-7
Traceback (most recent call last):
  File "positional-params-4.py", line 18, in <module>
    print(foo(x=1, y=2, z=10))
TypeError: foo() got some positional-only arguments passed as keyword arguments: 'x, y'
```

V dalším kroku se pokusme pseudoparametr / posunout ještě více doleva, konkrétně mezi x a y:

```
# Poziční a čistě poziční parametry funkcí:
# - definice a volání funkce s jedním čistě pozičním parametrem
```

```
# funkce s jedním čistě pozičním parametrem
def foo(x, /, y, z):
    return x + y - z

# volání funkce s předáním argumentů
print(foo(1, 2, 10))

# explicitní specifikace parametru y a z jménem
# je povolena
print(foo(1, 2, z=10))
print(foo(1, y=2, z=10))

# explicitní specifikace parametru x jménem
# není povolena (nezávisle na pořadí)
print(foo(x=1, y=2, z=10))
print(foo(z=1, y=2, x=10))
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/positional-params/positional-params-5.py

Nyní budou první tři volání funkce korektní, ale čtvrtý a pátý způsob již nikoli:

```
-7
-7
-7
```

Traceback (most recent call last):

File "positional-params-5.py", line 19, in <module>

print(foo(x=1, y=2, z=10))

TypeError: foo() got some positional-only arguments passed as keyword arguments: 'x'

4.3.5 Nekorektní umístění pseudoparametru /

Pseudoparametr / ovšem nesmí být uveden na prvním místě, což by ostatně nemělo ani žádný praktický význam. Je to totiž stejné, jakoby nebyl vůbec uveden.

```
# Poziční a čistě poziční parametry funkcí:
# - pokus o zápis funkce bez čistě pozičních parametrů
# - tato chyba je zachycena interpretrem Pythonu
```

— 4 Postupné rozšiřování syntaxe a sémantiky Pythonu

```
# chybná definice funkce bez čistě pozičních parametrů
def foo(/, x, y, z):
    return x+y-z

print(foo(1, 2, 10))
print(foo(z=1, y=2, x=10))
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/positional-params/positional-params-6.py

Při pokusu o deklaraci funkce, v níž je pseudoparametr / uveden na prvním místě, dojde k výjimce typu *SyntaxError*:

```
File "positional-params-6.py", line 4
    def foo(/, x, y, z):
            ^
SyntaxError: invalid syntax
```

U běžných pozičních parametrů platí, že jejich příslušné argumenty můžeme při volání funkce zadat v libovolném pořadí, ovšem pochopitelně v případě, že je zapíšeme se jménem:

```
# Poziční a čistě poziční parametry funkcí:
# - definice a volání funkce s jedním čistě pozičním parametrem

# funkce s jedním čistě pozičním parametrem
def foo(x, /, y, z):
    return x + y - z

# volání funkce s předáním argumentů
print(foo(1, 2, 10))

# argument pro parametr x se musí zadat pozičně
# ostatní argumenty již lze specifikovat se jménem
# parametru
print(foo(1, z=1, y=2))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/positional-params/positional-params-7.py

Výsledky ukazují korektní chování:

```
-7  
2
```

4.3.6 Definice výchozích hodnot u čistě pozičních parametrů

I u čistě pozičních parametrů lze v případě potřeby specifikovat jejich výchozí hodnotu. V případě, že argument pro tento parametr nebude při volání funkce uveden, bude použita tato výchozí hodnota. Python se tedy v tomto případě chová k čistě pozičním parametrům stejně, jako k běžným pozičním parametrům. Opět si to ukažme na jednoduchém demonstračním příkladu:

```
# Poziční a čistě poziční parametry funkcí:  
# - definice a volání funkce s jedním čistě pozičním parametrem  
#   a se specifikací výchozí hodnoty  
  
# funkce s jedním čistě pozičním parametrem  
# a specifikací výchozích hodnot  
def foo(x=0, /, y=0, z=0):  
    return x + y - z  
  
# volání funkce bez předání argumentů:  
# pro x, y, a z se použijí výchozí hodnoty  
print(foo())  
  
# volání funkce s předáním jediného argumentu:  
# pro y a z se použijí výchozí hodnoty  
print(foo(10))  
  
# volání funkce s předáním dvou argumentů:  
# pro z se použije výchozí hodnota  
print(foo(10, 20))  
  
# volání funkce s předáním všech tří argumentů  
print(foo(1, 2, 10))
```



```
# druhé dva argumenty lze stále specifikovat i  
# se zadáním jména parametru  
print(foo(1, z=1, y=2))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/positional-params/positional-params-8.py

Tento příklad je spuštěn bez chyby a měl by na terminál vypsát následující sekvenci výsledků:

```
0  
10  
30  
-7  
2
```

4.4 Mroží operátor

V rámci Pythonu verze 3.8 došlo ke dvěma rozšířením tohoto programovacího jazyka. O prvním rozšíření jsme se již zmínili; jednalo se o specifikaci těch parametrů funkcí, které je možné při volání funkce používat jen pozičně (tedy příslušný argument se předá bez možnosti explicitní specifikace jména parametru). A druhou novinkou, která se v Pythonu 3.8 objevila, je takzvaný *mroží operátor* (anglicky *walrus operator*). Tento název vychází z vizuální podoby tohoto operátoru, který se zapisuje dvojicí znaků `:=` (jde tedy o smajlík, který připomíná mroží kly). Přesný popis syntaxe a zejména sémantiky tohoto operátoru nalezneme v dokumentu PEP 572 - Assignment Expressions (zkratka *PEP* znamená *Python Enhancement Proposal*, tedy strukturovaně napsaný návrh na rozšíření jazyka).

Mroží operátor umožňuje provést operaci přiřazení v rámci *výrazu* (*expression*), což znamená, že se v první řadě jedná o rozšíření sémantiky Pythonu, protože před zavedením mrožího operátoru se přiřazení provádělo jen v rámci *příkazu* (*statement*). Zjednodušeně řečeno: operace přiřazení sama o sobě byla příkazem, a to i v případě, že se hodnota přiřazovala do většího množství proměnných (definice přiřazení je totiž rekurzivní). V souvislosti s přiřazením uvnitř výrazu s využitím mrožího operátoru se začal používat termín *named expression*.

4.4.1 Proměnná definovaná v podmínce

Podívejme se na jeden ze základních způsobů využití mrožího operátoru. Budeme chtít zkontrolovat, zda je heslo uložené v proměnné `password` dostatečně dlouhé. Pokud tomu tak není (tedy v případě, že heslo je příliš krátké) bude o tom uživatel informován zprávou „Password should

be longer than {length} chars“, kde se za „{length}“ dosadí zjištěná délka. To vlastně znamená, že vypočtenou délku hesla používáme na dvou místech, ovšem současně ji využijeme jen uvnitř programového bloku s podmínkou. V takovém případě si můžeme délku uložit do proměnné, která bude využita (čtena) právě jen v onom jediném programovém bloku. Pro tento účel využijeme mroží operátor:

```
# Mroží operátor v Pythonu:
# - mroží operátor zapsaný v podmínce

limit = 8

password = "Hello world"

if (length := len(password)) < limit:
    print(f"Password should be longer than {length} chars")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/walrus-operator/walrus-operator-1.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/walrus-operator/walrus-operator-1.py)

Povšimněte si, že inicializace proměnné přiřazením je zapsána do kulatých závorek.

Pro kratší heslo (= řetězec uložený v proměnné password) by se mělo vypsát varování. Můžeme si to snadno otestovat:

```
# Mroží operátor v Pythonu:
# - mroží operátor zapsaný v podmínce

limit = 8

password = "Hello"

if (length := len(password)) < limit:
    print(f"Password should be longer than {length} chars")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/walrus-operator/walrus-operator-2.py

Po spuštění tohoto skriptu by se mělo vypsát následující varování:

```
Password should be longer than 5 chars
```


4.4.2 Vynechání kulatých závorek okolo přiřazení s mrožím operátorem

Kulaté závorky okolo přiřazení s mrožím operátorem nejsou ve skutečnosti vyžadována z pohledu *syntaxe*, takže je můžeme vynechat, aniž by došlo k detekci chyby interpretrem Pythonu. Ovšem kvůli tomu, že má mroží operátor nejnižší prioritu, je většinou nutné závorky ve zdrojových kódech použít. Ostatně si otestujme, co se stane v případě, že závorky vynecháme:

```
# Mroží operátor v Pythonu:
# - sémanticky chybné použití mrožího operátoru (bez závorek)

limit = 8

password = "Hello"

if length := len(password) < limit:
    print(f"Password should be longer than {length} chars")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-operator-error-1.py

Po spuštění takto upraveného skriptu se vypíše zpráva, která naznačuje, že do proměnné `length` se uložil výsledek porovnání délky hesla s proměnnou `limit` (tedy pravdivostní hodnota), což je ovšem něco jiného, než samotná délka hesla:

```
Password should be longer than True chars
```

4.4.3 Platnost proměnné inicializované mrožím operátorem

Podle typického způsobu umístění výrazu s mrožím operátorem by se mohlo zdát, že proměnná, která je tímto způsobem vytvořená a inicializovaná, bude mít platnost jen uvnitř příslušného programového bloku (tedy například bloku `if` či bloku se smyčkou atd.). Zkusme si tento předpoklad ověřit spuštěním příkladu, v němž budeme číst obsah proměnné `length` i vně bloku s `if`:

```
# Mroží operátor v Pythonu:
# - mroží operátor zapsaný v podmínce,
# - čtení proměnné za programovým blokem

limit = 8

password = "Hello"
```

```
if (length := len(password)) < limit:
    print(f"Password should be longer than {length} chars")

print(f"Password length: {length} chars")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-operator-3.py

Spuštění tohoto skriptu nepovede k žádné běhové výjimce typu „přístup k neexistující proměnné“ (`NameError`), což znamená, že proměnná `length` je viditelná i vně programového bloku:

```
Password should be longer than 5 chars
Password length: 5 chars
```

Poznámka: toto chování Pythonu může být zpočátku poněkud matoucí.

4.4.4 Vyřešení problému opakovaných výpočtů

Jak jsme již mohli vidět v předchozích příkladech, umožňuje nám mroží operátor eliminovat opakované volání stejných funkcí či vyhodnocování stejných výrazů. Zkusme si tedy tento koncept rozvinout. V následujícím skriptu se dvakrát volají funkce `len` a `sum` pro stejné argumenty, což je pochopitelně zbytečné a zdržující:

```
# Mroží operátor v Pythonu:
# - dvojí volání funkcí len() a sum()

values = (1, 2, 3, 4, 5)

result = {
    "count": len(values),
    "sum": sum(values),
    "mean": sum(values) / len(values),
}

print(result)
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-operator-4.py

Výsledkem skriptu by měl být slovník s následujícím obsahem – počtem prvků, součtem prvků a vypočteným průměrem:

```
{'count': 5, 'sum': 15, 'mean': 3.0}
```

Samozřejmě je možné tento problém vyřešit tak, že explicitně vytvoříme nové pomocné proměnné nazvané například `count` a `summ`, které použijeme ve výpočtech a tím eliminujeme opakované provádění stejných výpočtů:

```
# Mroží operátor v Pythonu:
# - volání funkcí len() a sum() pouze jedenkrát,
# - použití pomocných proměnných na globální úrovni

values = (1, 2, 3, 4, 5)

count = len(values)
summ = sum(values)

result = {
    "count": count,
    "sum": summ,
    "mean": summ / count,
}

print(result)
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-operator-5.py

Se stejným výsledkem:

```
{'count': 5, 'sum': 15, 'mean': 3.0}
```

Můžeme ovšem namísto toho použít mroží operátor a samotné vytvoření a inicializaci pomocných proměnných přenést do samotného výpočtu:

```
# Mroží operátor v Pythonu:
# - volání funkcí len() a sum() pouze jedenkrát,
# - použití mrožího operátoru

values = (1, 2, 3, 4, 5)

result = {
    "count": (count := len(values)),
    "sum": (summ := sum(values)),
    "mean": summ / count,
}

print(result)
```

Zdrojový kód tohoto příkladu můžete najít na adrese
[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/walrus-operator/walrus-operator-6.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/walrus-operator/walrus-operator-6.py)

```
{'count': 5, 'sum': 15, 'mean': 3.0}
```

4.4.5 Mroží operátor a programová smyčka typu `while`

Mroží operátor se mnohdy s výhodou používá společně s programovou smyčkou typu `while`. Konkrétně se to týká situace, kdy je nutné před zahájením další iterace provést nějakou operaci a následně testovat její výsledek. Poměrně typickým příkladem může být programová smyčka, která bude číst obsah textového souboru, a to stylem „řádek po řádku“. Ve chvíli, kdy se detekuje prázdný řádek (což musí být poslední řádek v souboru) nebo se skutečně dosáhne konce souboru, je smyčka ukončena. Pokud by se nepoužil mroží operátor, je nutné v programovém kódu dvakrát použít volání metody `readline`, protože je nutné korektně reagovat i na situaci, kdy je soubor prázdný nebo je jeho první řádek prázdný:

```
# Mroží operátor v Pythonu:
# - programová smyčka typu while realizovaná
#   bez použití mrožího operátoru

# otevření souboru pro čtení
with open("test.txt", "r") as f:
    # přečtení prvního řádku ze souboru
    line = f.readline()
    # přečtení dalších řádků ze souboru
    while line:
        print(line.strip())
        line = f.readline()
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-loop-1.py

Právě v takových situacích je vhodné použít mroží operátor, který nám umožní zapsat celou logiku s programovou smyčkou stylem, který připomíná řešení používaná v programovacím jazyku C:

```
# Mroží operátor v Pythonu:
# - programová smyčka typu while založená
#   na použití mrožího operátoru

# otevření souboru pro čtení
with open("test.txt", "r") as f:
    # přečtení všech řádků ze souboru
    while line := f.readline():
        print(line.strip())
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-loop-2.py

4.4.6 Mroží operátor a generátorová notace

S mrožím operátorem se setkáme taktéž při zápisu generátorových notací (k těm se vrátíme později). Podívejme se například na program, který vypočítá druhé mocniny hodnot předaných ve vstupním seznamu, ovšem pouze za předpokladu, že tyto druhé mocniny budou menší než zadaná mezní hodnota 30:

```
# Mroží operátor v Pythonu:
# - dvojí vyhodnocení výrazu num * num pro každý prvek
#   ze vstupního seznamu

numbers = [1, 9, 2, 8, 3, 7, 4, 6, 5]

squares = [num * num for num in numbers if num * num < 30]

print(squares)
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-list-comprehension-1.py

Vypočtený výsledek:

```
[1, 4, 9, 16, 25]
```

Povšimněte si ovšem, že vlastní výpočet druhé mocniny (`num * num`) musel být proveden dvakrát – jak pro test na překročení mezní hodnoty, tak i pro přidání nového prvku do vytvářeného seznamu. A právě pro eliminaci dvojího výpočtu můžeme použít mroží operátor. Upravený příklad provádějící shodný výpočet bude vypadat následovně:

```
# Mroží operátor v Pythonu:
# - využití mrožího operátoru: výraz num * num se
#   vyhodnotí pro každý prvek jen jedenkrát

numbers = [1, 9, 2, 8, 3, 7, 4, 6, 5]

squares = [squared for num in numbers if (squared := num * num) < 30]

print(squares)
```

Zdrojový kód tohoto příkladu můžete najít na adrese [https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/walrus-operator/walrus-list-comprehension-2.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/walrus-operator/walrus-list-comprehension-2.py)

Výsledky budou totožné, jako u předchozího příkladu.

Ještě ve větší míře bude zjednodušení výpočtů patrné u skriptu, který počítá Ackermannovu funkci a vrací ty výsledky, které jsou liché. Samotná Ackermannova funkce je navržena takovým způsobem, aby byl její výpočet velmi náročný a proto je urychlení výpočtů zřejmé.

Bez použití mrožího operátoru:

```
# Mroží operátor v Pythonu:
# - dvojí volání Ackermannovy funkce pro
#   každou dvojici parametrů

from time import perf_counter

def A(m, n):
    """Ackermannova funkce."""
    if m == 0:
```

```
        return n + 1
    if n == 0:
        return A(m - 1, 1)
    return A(m - 1, A(m, n - 1))
```

```
start = perf_counter()
```

```
results = [A(m, n) for m in range(0, 4) for n in range(0, 7) if A(m, n) % 2 != 0]
```

```
print(results)
```

```
finish = perf_counter()
```

```
print(f"Duration: {finish-start} seconds")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/walrus-operator/walrus-list-comprehension-3.py

Výsledky:

```
[1, 3, 5, 7, 3, 5, 7, 3, 5, 7, 9, 11, 13, 15, 5, 13, 29, 61, 125, 253, 509]
```

```
Duration: 0.046124089043587446 seconds
```

S použitím mrožího operátoru:

```
# Mroží operátor v Pythonu:
```

```
# - optimalizované volání Ackermannovy funkce
```

```
# pro každou dvojici parametrů.
```

```
from time import perf_counter
```

```
def A(m, n):
    """Ackermannova funkce."""
    if m == 0:
        return n + 1
    if n == 0:
        return A(m - 1, 1)
    return A(m - 1, A(m, n - 1))
```

```
# zjistit čas před výpočtem
start = perf_counter()

results = [result for m in range(4) for n in range(7) if (result := A(m, n)) % 2 != 0]

print(results)

# zjistit čas po výpočtu
finish = perf_counter()

# zobrazení celkové doby výpočtu
print(f"Duration: {finish-start} seconds")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/walrus-operator/walrus-list-comprehension-4.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/walrus-operator/walrus-list-comprehension-4.py)

Výsledky:

```
[1, 3, 5, 7, 3, 5, 7, 3, 5, 7, 9, 11, 13, 15, 5, 13, 29, 61, 125, 253, 509]
Duration: 0.027164419880136847 seconds
```

4.4.7 Hodnota pomocné proměnné po dokončení výpočtů

I při zápisu výpočtu založeného na generátorové notaci platí, že pomocná proměnná vytvořená a inicializovaná s využitím mrožího operátoru (což je v našem případě proměnná `result`) zůstane viditelná i po dokončení výpočtu. Můžeme se o tom snadno přesvědčit – po spuštění dalšího příkladu se vypíšou jak vypočtené hodnoty, tak i poslední hodnota přiřazená do proměnné `result`:

```
# Mroží operátor v Pythonu:
# - lokální proměnná result je viditelná
# i po konstrukci seznamu s výsledky.
```

```
def A(m, n):
    """Ackermannova funkce."""
    if m == 0:
        return n + 1
    if n == 0:
        return A(m - 1, 1)
    return A(m - 1, A(m, n - 1))
```



```
results = [result for m in range(4) for n in range(7) if (result := A(m, n)) % 2 != 0]

# zobrazení vypočtených výsledků
print(results)

# existuje proměnná result?
print(result)
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/walrus-operator/walrus-list-comprehension-5.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/walrus-operator/walrus-list-comprehension-5.py)

```
[1, 3, 5, 7, 3, 5, 7, 3, 5, 7, 9, 11, 13, 15, 5, 13, 29, 61, 125, 253, 509]
509
```

4.4.8 Příloha: prioritá operátorů

Mroží operátor má (alespoň prozatím) nejnižší prioritu. Jeho přidáním do jazyka Python došlo k rozšíření tabulky operátorů. Seřadme si pro přehlednost všechny existující operátory podle jejich priority:

Pořadí	Operátory
1.	(výrazy...), [výrazy...], {klíč: hodnota...}, {výrazy...}
2.	x[index], x[index:index], x(argumenty...), x.atribut
3.	await x
4.	**
5.	+x, -x, ~x (unární)
6.	*, @, /, //, %
7.	+, -
8.	<<, >>
9.	&
10.	^
11.	
12.	in, not in, is, is not, <, <=, >, >=, !=, ==
13.	not x
14.	and
15.	or
16.	if - else
17.	lambda
18.	:=

4.5 Generické datové typy

Do Pythonu verze 3.9 byla přidána podpora pro specifikaci generických datových typů. Jedná se o datové typy, které mohou být takzvaně *parametrizovatelné* a z tohoto důvodu se někdy setkáváme s označením *parametric type* (tedy parametrické typy, což je ovšem poněkud zavádějící). Příkladem takového datového typu je množina (`set`), seznam (`list`), nebo slovník (`dict`). Volbou parametru či parametrů můžeme z generického datového typu odvodit instanci, která může obsahovat pouze prvky určitého typu. Takový datový typ se někdy označuje termínem *parameterized type* (což je odlišné označení o *parametric type*, to již ovšem zabíháme do možná zbytečných podrobností). Příkladem konkrétní instance generického datového typu slovník (`dict`) může být `dict[str, Complex]`, tj. slovník, jehož klíči mohou být pouze řetězce a hodnotami komplexní čísla (instance třídy `Complex`).

4.5.1 Standardní kolekce

V rámci tohoto rozšíření Pythonu 3.9 se několik standardních kolekcí (především pak seznam, n-tice, množina, slovník atd.) stalo generickými typy. Týká se to těchto standardních kolekcí:

- `tuple`
- `list`
- `dict`
- `set`
- `frozenset`

Navíc se to nově týká i typu, který se jmenuje `type` (což může být opět matoucí).

4.5.2 Další generické kolekce

To ale není vše, protože se tato změna týká i dalších typů kolekcí ze standardní knihovny Pythonu. Je jich celá řada, konkrétně:

- `collections.deque`
- `collections.defaultdict`
- `collections.OrderedDict`
- `collections.Counter`
- `collections.ChainMap`
- `collections.abc.Awaitable`
- `collections.abc.Coroutine`
- `collections.abc.AsyncIterable`
- `collections.abc.AsyncIterator`
- `collections.abc.AsyncGenerator`

- `collections.abc.Iterable`
- `collections.abc.Iterator`
- `collections.abc.Generator`
- `collections.abc.Reversible`
- `collections.abc.Container`
- `collections.abc.Collection`
- `collections.abc.Callable`
- `collections.abc.Set`
- `collections.abc.MutableSet`
- `collections.abc.Mapping`
- `collections.abc.MutableMapping`
- `collections.abc.Sequence`
- `collections.abc.MutableSequence`
- `collections.abc.ByteString`
- `collections.abc.MappingView`
- `collections.abc.KeysView`
- `collections.abc.ItemsView`
- `collections.abc.ValuesView`
- `contextlib.AbstractContextManager`
- `contextlib.AbstractAsyncContextManager`
- `re.Pattern`
- `re.Match`

4.5.3 Starší typové aliasy, které mohou být odstraněny

Některé starší typové aliasy již nejsou díky výše popsanému vylepšení Pythonu zapotřebí a z tohoto důvodu byly označeny jako *deprecated*, což znamená, že sice nejsou z Pythonu odstraněny (až na `typing.Text`), ale k jejich odstranění může v budoucnosti dojít. Nicméně všechny dnes používané verze CPythonu tyto typové aliasy používají a používá je i dále popsaný nástroj *Mypy*. Jedná se o tyto třídy:

- `typing.Dict`
- `typing.List`
- `typing.Set`
- `typing.FrozenSet`
- `typing.Tuple`
- `typing.DefaultDict`
- `typing.OrderedDict`
- `typing.ChainMap`
- `typing.Counter`
- `typing.Deque`

- `typing.Pattern`
- `typing.Match`
- `typing.Text`
- `typing.AbstractSet`
- `typing.ByteString`
- `typing.Collection`
- `typing.Container`
- `typing.ItemsView`
- `typing.KeysView`
- `typing.Mapping`
- `typing.MappingView`
- `typing.MutableMapping`
- `typing.MutableSequence`
- `typing.MutableSet`
- `typing.Sequence`
- `typing.ValuesView`

4.5.4 Další informace o typovém systému programovacího jazyka Python

Bližší informace o generických datových typech si uvedeme v páté kapitole, v níž se budeme podrobněji věnovat typovému systému Pythonu.

4.6 Pattern matching

V relativně nedávno vydaném Pythonu verze 3.10 se objevila dlouho očekávaná novinka – takzvaný *strukturální pattern matching*. Jedná se o v jazyku Python zcela novou programovou konstrukci, která velmi vzdáleně připomíná konstrukci `switch-case` z programovacího jazyka C (odkud byla převzata do dalších programovacích jazyků, včetně C++ či Javy). Ovšem strukturální *pattern matching* ve skutečnosti programátorům nabízí mnohem více možností než původní značně primitivní konstrukce `switch-case`. Některé z těchto možností si ukážeme v této kapitole a pro úplnost si řekneme, jak a kde se *pattern matching* vlastně začal v informatice používat. Poměrně zajímavá by pro programátory mohla být i zmínka o programovacím jazyce *Coconut*, ve kterém se strukturální *pattern matching* objevil již před relativně dlouhou dobou a navíc je tento v mnoha ohledech zajímavý programovací jazyk transpilován (tedy transformován) přímo do Pythonu.

Poznámka: tato podkapitola bude v rámci čtvrté kapitoly nejdelší. Je tomu tak z toho důvodu, že strukturální pattern matching je skutečně velmi důležitým a užitečným rozšířením Pythonu a je dobré ho znát a začít používat v praxi.

4.6.1 Od SNOBOLu k Pythonu

Strukturální pattern matching je velmi důležitou konstrukcí, která za sebou má mnoho let vývoje. Nejedná se tedy ani zdaleka o „výkřik moderní doby“, ale naopak o známý koncept, který se ovšem až v posledních letech dostává i do mainstreamových jazyků, mezi něž Python samozřejmě patří. Koncept pattern matchingu se postupně vyvíjel a my budeme tento vývoj alespoň ve stručnosti sledovat v této podkapitole.

SNOBOL: programovací jazyk založený primárně na zpracování textových vzorů

Prvním významným jazykem s pattern matchingem (kde je dokonce pattern matching ústředním a neoddělitelným prvkem jazyka) je programovací jazyk *SNOBOL*. Samotný název *SNOBOL* je zkratkou odvozenou z *StriNg Oriented and symBolic Language*. Slavné Bellovy laboratoře jsou v oblasti elektroniky a samozřejmě taktéž informatiky velmi známou institucí. Ostatně právě v Bellových laboratořích byl zkonstruován první tranzistor (William Shockley, Gerald Pearson, John Bardeen, Walter Brattain), vytvořena první verze dodnes široce používaného programovacího jazyka *C* (Dennis Ritchie) a v neposlední řadě taktéž funkční prototyp operačního systému *UNIX* (Ken Thompson, Dennis Ritchie, Brian Kernighan, Douglas McIlroy, Joe Ossanna a další).

Kromě těchto známých projektů, z nichž například vynález tranzistoru byl odměněn Nobelovou cenou, však v Bellových laboratořích vzniklo i poměrně velké množství dalších více či méně úspěšných projektů z oblasti informatiky (ostatně operační systém *UNIX* i programovací jazyk *C* jsme již zmínili). Jedním z těchto projektů, který je dnes již poněkud pozapomenut, je právě i programovací jazyk nazvaný *SNOBOL*, který byl v Bellových laboratořích vyvíjen v průběhu let 1962 až 1967. Jeho autory jsou David J. Farber, Ralph E. Griswold a Ivan P. Polonsky.

Syntaxe programovacího jazyka *SNOBOL* je velmi jednoduchá, protože tento jazyk ve své základní podobě neobsahuje žádné podmíněné příkazy, programové smyčky ani jiné jazykové konstrukce známé z jiných (strukturovaných) jazyků. Namísto toho se struktura programů nejvíce podobá zápisu stavů a přechodů konečného automatu (*finite state machine*), což je ostatně při zpracovávání řetězců (či jiných dat) jeden ze základních nástrojů prakticky každého programátora.

Jen pro zajímavost se podívejme, jak vypadaly programy psané ve *SNOBOLu*. Příklad byl získán z Wikipedie:

```

OUTPUT = "What is your name?"
Username = INPUT
Username "J"                                :S(LOVE)
Username "K"                                :S(HATE)
MEH     OUTPUT = "Hi, " Username             : (END)
LOVE    OUTPUT = "How nice to meet you, " Username : (END)
HATE    OUTPUT = "Oh. It's you, " Username
END
```


Od SNOBOLu k doménově specifickému jazyku AWK

Samotný SNOBOL byl sice používán, a to poměrně intenzivně (typicky nikoli profesionálními programátory, ale odborníky z jiných oblastí, kteří potřebují resp. potřebovali zpracovávat textová data), ovšem z dnešního pohledu je mnohem důležitější fakt, že SNOBOL ovlivnil sémantiku dalšího programovacího jazyka. Jedná se o doménově specifický jazyk *AWK*, jenž je používán dodnes, typicky pro tvorbu pomocných skriptů pro zpracování či transformaci textových dat. Mnoho algoritmů je možné s využitím *AWK* realizovat velmi efektivním způsobem, a to právě díky (prakticky implicitnímu) *pattern matchingu*. A opět platí (což asi není náhoda), že *AWK* je mnohdy používán tou samou skupinou uživatelů, jako tomu bylo v případě SNOBOLu.

Strukturální pattern matching v programovacím jazyku ML

Koncept *pattern matchingu*, který byl představen v šedesátých letech ve SNOBOLu, by dále rozvinut v programovacím jazyku ML, odkud se dále rozšířil do dalších programovacích jazyků: na přímé linii především do jazyka Caml a OCaml (a vlastně i do programovacího jazyka F#, který je v přímé linii ML) a na nepřímé linii do Rustu, Pythonu 3.10 atd. Podívejme se nyní ve stručnosti na praktické použití této programovací techniky právě v programovacím jazyku ML.

Obecný zápis funkce s pattern matchingem vypadá následovně:

```
fun <jméno> <vzorek> = <tělo/výraz>
```

Většinou se však v praxi používá větší množství vzorků, které jsou spojeny znakem „|“ neboli „or“. V takovém případě jsou vzorky postupně procházeny a pokud budou vyhovovat předaným datům, bude příslušná větev funkce vykonána:

```
fun <jméno> <vzorek> = <tělo/výraz>
| <jméno> <vzorek> = <tělo/výraz>
| <jméno> <vzorek> = <tělo/výraz>
| <jméno> <vzorek> = <tělo/výraz>
```

Naproti tomu příkladem běžné funkce bez pattern matchingu je funkce `length` v této podobě (pro seznamy):

```
fun length(x) = if null(x) then 0
                else 1 + length(tl(x));
```

V této funkci se provede vždy jedna z větví na základě zapsané podmínky (*case analysis*). Ovšem díky existenci pattern matchingu lze stejnou funkci zapsat odlišně – a to vyjmenováním takzvaných *invariantů* algoritmu: